Kisan AI — App Flow Reference

Keep this file in the project root (or `.qoder/rules`) so Qoder has full context on every request.

1. Screen Map

```
Welcome
  └─ Get Started ──► Farm Location
  └─ Skip demo ─────► Dashboard (demo/mock
data)

Farm Location (Province → District →
City)
  └─ Continue ──► Language Selection
  └─ Back ──────► Welcome

Language Selection (Urdu / Punjabi / Sindhi
/ Pashto / Balochi)
  └─ Continue ──► Farmer Type
  └─ Back ──────► Farm Location

Farmer Type (New Farmer / Experienced
Farmer — single select)
  └─ Continue ──► Crops
  └─ Back ──────► Language Selection

Crops (multi-select grid)
```

```
  └─ Continue ──▶ Livestock
  └─ Back ──────▶ Farmer Type

Livestock (multi-select grid)
  └─ Continue ──▶ Farm Size
  └─ Back ──────▶ Crops

Farm Size (stepper + unit)
  └─ Continue ──▶ POST
/api/users/onboarding ──▶ Dashboard
  └─ Back ──────▶ Livestock

Dashboard (home)
  ├─ Weather card
  ├─ Ask Kisan AI (chat entry)
  ├─ Crop Disease Detection ──▶
Camera/Gallery ──▶ Preview ──▶ Analyzing
──▶ Disease Result ──▶ Pesticide
Recommendation
  ├─ Pest & Insect Detection ──▶
Camera/Gallery ──▶ Preview ──▶ Analyzing
──▶ Pest Result ──▶ Insecticide
Recommendation
  ├─ Crop Recommendation ──▶ Result List
  ├─ Irrigation Guide ──▶ Result Card
  └─ Hamburger Menu ──▶ Profile / History
/ Settings / Logout
```

2. Design System (reference — see Chunk 1 prompt for full detail)

- Background: `#F5F0E4` (cream)

- Primary: `#1F7A3D` (forest green)
- Heading text: `#0F1B0F`
- Body text: `#6B7568`
- Component style: large radius (16–20px), pill buttons, green border + tint on selected state
- Shared shell: `OnboardingScaffold` with 7-segment `StepProgressBar` + fixed `Back` / `Continue` row

3. Result Screen Contract (applies to Disease, Pest, Pesticide, Insecticide)

Every result screen must answer, in this order:

1. **What is the problem?** — identified crop/pest + confidence
2. **Why did it happen?** — causes/conditions
3. **What should I do now?** — recommendation/treatment
4. **What should I avoid?** — safety precautions / warnings

Below 0.70 confidence → show fallback card: "Consult an agronomist" + top-2 candidate guesses, not a forced single answer.

4. Data Flow (ties to ERD)

- Onboarding submit → `USERS` + `PLOTS` (+ `PLOT_CROPS`, `PLOT_LIVESTOCK`) created
- Photo upload → `IMAGES` row created, file goes to OSS, `upload_type` = `disease` or `pest`
- AI inference result → `DISEASE_DETECTIONS` or `PEST_DETECTIONS` row linked to that image
- Confidence ≥ 0.70 and problem detected → Rules Engine writes `PESTICIDE_RECOMMENDATIONS` or `INSECTICIDE_RECOMMENDATIONS`, checking `WEATHER_CACHE` first (`weather_blocked` flag set if unsafe)
- Crop Recommendation request → reads `PLOTS` (soil/season/water inputs) → writes `CROP_RECOMMENDATIONS`
- Every screen the farmer views gets logged to `ANALYSIS_HISTORY` for the History screen

5. Database Security Rules (mandatory — apply to every endpoint)

- **Never build raw SQL strings from user input.** Use the SQLAlchemy ORM (`.filter()`, `.where()`) for all queries — it parameterizes automatically.
- **If raw SQL is ever unavoidable**, use bound parameters only:

```
# Correct
db.execute(text("SELECT * FROM users
WHERE email = :email"), {"email":
user_email})

# Never do this
db.execute(f"SELECT * FROM users WHERE
email = '{user_email}'")
```

- **Every request body must pass through a Pydantic schema first** — type-check and length-validate before it ever reaches the database layer.
- **Backend RDS user runs with least privilege** — no `DROP` / `ALTER` permissions on the connection the app uses day-to-day.
- **Passwords are hashed** (bcrypt/argon2) — `password_hash` column only, never plaintext.
- **All chemical/dosage output still comes only from the Rules Engine**, never written or altered by raw user input or by the AI model directly (ties to the Golden Rule in Section 3).

6. Naming Conventions (for Qoder to follow consistently)

- Flutter widgets: PascalCase, one widget per file (`onboarding_scaffold.dart`)

- API routes: `/api/{module}/{action}` (e.g. `/api/disease/detect`, `/api/weather/current`)
- Backend modules match the table groups above: `auth`, `plots`, `images`, `disease`, `pests`, `pesticides`, `insecticides`, `weather`, `irrigation`, `chat`, `history`